MP5: Kernel-Level Thread Scheduling

Mauricio Aguilar Alonso
UIN: 434002903
CSCE 410/611: Operating Systems

Assigned Tasks

Main assignment: Completed. FIFO scheduling and terminating thread functions.
Bonus option 1: Interrupt management. Not completed.
Bonus option 2: Round-robin scheduling. Not completed.
Bonus option 3: Processes with separate address spaces. Not completed.

System Design

The scheduler decides which ready thread gets the CPU next. It keeps a first-in, first-out (FIFO) queue of pointers to thread control blocks. When a thread gives another thread a turn and still has work to do, it joins the tail of the queue. The thread at the head gets the next turn. For example, if thread 1 gives up the CPU while threads 2, 3, and 4 are waiting, thread 2 runs and the queue holds threads 3, 4, and 1 in that order.

A thread that finishes gives up the CPU permanently. It is still using its stack when its thread function returns, so another thread must release that stack after the context switch. The scheduler remembers the finished thread until cleanup is safe. If the ready queue is empty, an idle thread provides a stack on which execution can continue.

Code Description

`scheduler.H` defines the ready queue node and adds pointers to the head and tail of the queue, the idle thread, and the finished thread awaiting cleanup. It also declares the two helpers used for shutdown and cleanup.

```
class Scheduler {

   /* Ready Queue members*/
   struct Node {
      Thread * thread;
      Node * next;
   };
   Node * ready_head;
   Node * ready_tail;
   Thread * idle;
   Thread * finished;
   void collect_finished();

public:
   static void terminate_current();
   /* Entry from thread_shutdown when a thread function returns. */
```

Ready queue members and helper declarations in `scheduler.H`.

`thread.H` declares the thread destructor so deleting a finished thread also releases its stack.

The constructor and the four methods for adding, resuming, yielding, and terminating threads are listed in the handout. The entries below identify the additional functions used in this implementation.

scheduler.C: Scheduler() The constructor sets the head and tail to null because the ready queue starts empty. It records the active scheduler so a returning thread can reach it during shutdown.

```
Scheduler::Scheduler() : ready_head(0), ready_tail(0), idle(0), finished(0) {
  active_scheduler = this;
  Console::puts("Constructed Scheduler.\n");
}
```

Initializing the scheduler.

scheduler.C: resume() A thread that becomes ready goes to the tail of the queue. If the queue is empty, its node becomes both the head and the tail.

```
void Scheduler::resume(Thread * _thread) {
  /* A thread may only appear in the ready queue once. */
  assert(_thread);
  for (Node * queued = ready_head; queued; queued = queued->next) {
    assert(queued->thread != _thread);
  }
  Node * node = new Node{_thread, 0};
  if (ready_tail) {
    ready_tail->next = node;
  } else {
    ready_head = node;
  }
  ready_tail = node;
}
```

Appending a thread to the ready queue.

scheduler.C: add() A newly created thread enters the queue at the tail. This method calls `resume()` because it already performs that operation.

```
void Scheduler::add(Thread * _thread) {
  resume(_thread);
}
```

Adding a newly created thread.

scheduler.C: yield() The scheduler removes the head of the ready queue and dispatches that thread if it differs from the running thread. If the queue is empty, the scheduler gives the CPU to the idle thread.

```
void Scheduler::yield() {
  collect_finished();
  if (!ready_head) {
    /* Keep the idle thread outside the FIFO queue. */
    if (!idle) idle = new Thread(run_idle, new char[1024], 1024);
    if (Thread::CurrentThread() != idle) Thread::dispatch_to(idle);
    return;
  }

  Node * node = ready_head;
  Thread * next = node->thread;
  ready_head = node->next;
  if (!ready_head) ready_tail = 0;
  delete node;

  if (next != Thread::CurrentThread()) {
    Thread::dispatch_to(next);
  }
}
```

Selecting the next thread to run.

scheduler.C: run_idle() *Added for this implementation.* When the last runnable thread finishes, the CPU needs another stack to switch to. The idle thread provides that stack and repeatedly yields so any ready thread can run.

```
static void run_idle() {
  for (;;) active_scheduler->yield();
}
```

The idle thread function.

thread.C: thread_shutdown() The initial stack contains this function's address. When the thread function returns, execution continues here and the shutdown code asks the scheduler to terminate the running thread.

```
static void thread_shutdown() {
    /* setup_context placed this return address above the thread function. */
    Scheduler::terminate_current();
}
```

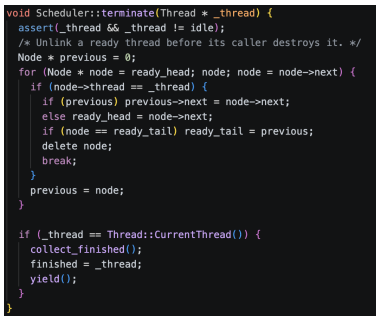
Entering shutdown when a thread function returns.

scheduler.C: terminate_current() *Added for this implementation.* The shutdown function receives no arguments. This method looks up the running thread and passes it to the active scheduler for termination.

```
void Scheduler::terminate_current() {
  assert(active_scheduler);
  active_scheduler->terminate(Thread::CurrentThread());
}
```

Passing the running thread to the scheduler.

scheduler.C: terminate() If the given thread is queued, the scheduler unlinks its node and updates the head or tail as needed. A thread terminating itself first collects any previously finished thread, saves its own pointer in `finished`, and gives up the CPU permanently.



```
void Scheduler::terminate(Thread * _thread) {
  assert(_thread && _thread != idle);
  /* Unlink a ready thread before its caller destroys it. */
  Node * previous = 0;
  for (Node * node = ready_head; node; node = node->next) {
    if (node->thread == _thread) {
      if (previous) previous->next = node->next;
      else ready_head = node->next;
      if (node == ready_tail) ready_tail = previous;
      delete node;
      break;
    }
    previous = node;
  }

  if (_thread == Thread::CurrentThread()) {
    collect_finished();
    finished = _thread;
    yield();
  }
}
```

Removing a thread from scheduling.

scheduler.C: collect_finished() *Added for this implementation.* A thread still needs its stack while giving up the CPU. This helper checks that a different thread is running before deleting the finished thread. One pointer is enough because each exiting thread collects its predecessor before taking its place.

```
void Scheduler::collect_finished() {
  /* Only allow a thread to release its predecessor only after switching stacks. */
  if (finished && finished != Thread::CurrentThread()) {
    delete finished;
    finished = 0;
  }
}
```

Checking that execution has switched to another stack.

thread.C: `~Thread()` *Added for this implementation.* The kernel allocates each thread's stack separately. This destructor releases that stack with `delete[]` when the finished thread is deleted. The supplied memory pool accepts the release request but does not reuse the memory.



Releasing the finished thread's stack.

Testing

Testing used the `_USES_SCHEDULER_` and `_TERMINATING_FUNCTIONS_` macros in `kernel.C`.